

CONCETTI AVANZATI E REACT HOOKS (Parte 1)

Che cosa sono gli Hooks

Definizione. Gli hook sono **funzioni speciali di React che permettono a un componente-funzione di "agganciarsi" a funzionalità interne di React**: memoria persistente tra i render, effetti collaterali, riferimenti al DOM, accesso a dati condivisi.

Perché esistono — il contesto storico. Prima del 2019 esistevano due tipi di componenti: le *class component*, che potevano avere stato e ciclo di vita, e le *function component*, che erano "stupide" e potevano solo mostrare props. Se un componente iniziava semplice e poi aveva bisogno di stato, dovevi riscriverlo interamente come classe. Gli hook hanno eliminato questa distinzione: oggi si scrivono solo function component, e le classi sono legacy.

Il nome "hook" (gancio) è letterale: `useState` aggancia il tuo componente al sistema di stato di React, `useEffect` lo aggancia al ciclo di vita del render.

Tutti gli hook di React iniziano con `use`. Non è una decorazione: è la convenzione su cui si basano il linter e le regole che seguono.

useState — l'errore base (perché lo stato serve)

Il corso non parte dalla soluzione, parte dal **problema**. È l'approccio migliore, quindi lo riproduco.

Prova a cambiare una variabile normale:

```
const Componente = () => {  
  let titolo = "Hello World";  
  
  const cambiaTitolo = () => {  
    titolo = "Nuovo Titolo";  
    console.log(titolo); // stampa "Nuovo Titolo"  
  };  
  
  return (  
    <>  
      <h2>{titolo}</h2>  
      <button onClick={cambiaTitolo}>Change me</button>  
    )  
  );  
}
```

```
</>  
);  
};
```

Clicchi il bottone. In console appare "Nuovo Titolo". **Ma sullo schermo resta scritto "Hello World"**.

Perché. Due problemi distinti, che la trascrizione separa bene:

1. **React non sa che qualcosa è cambiato.** Modificare una variabile JavaScript non avvisa nessuno. React ridisegna un componente solo quando gli viene detto esplicitamente che deve farlo. Nessun nuovo render → nessun aggiornamento a schermo.
2. **Anche se ci fosse un nuovo render, il valore andrebbe perso.** Un componente è una funzione: a ogni render viene rieseguita da capo, e la riga `let titolo = "Hello World"` reinizializzerebbe la variabile al valore di partenza. Una variabile locale **non ha memoria tra un render e l'altro**.

Servono quindi due cose insieme: **un valore che sopravvive ai render** e **un meccanismo che avvisa React quando cambia**. Questo è esattamente `useState`.

useState — l'hook

Definizione. `useState` è l'hook che dichiara una variabile di **stato**: un valore che React conserva tra i render e il cui cambiamento provoca un nuovo render del componente.

```
import { useState } from 'react';  
  
const Componente = () => {  
  const [titolo, setTitolo] = useState("Hello World");  
  
  const cambiaTitolo = () => {  
    setTitolo("Nuovo Titolo");  
  };  
  
  return (  
    <>  
      <h2>{titolo}</h2>  
      <button onClick={cambiaTitolo}>Change me</button>  
    </>  
  );  
};
```

```
};
```

Ora funziona: il testo cambia a schermo.

Anatomia della riga. `useState` ritorna un array di due elementi, che si destruttura (ecco perché la destrutturazione della Lezione 2 era un prerequisito):

```
const [titolo, setTitolo] = useState("Hello World");  
  
//   ↑      ↑           ↑  
// valore  funzione     valore iniziale  
// attuale che lo aggiorna (solo al primo render)
```

- **Primo elemento:** il valore corrente. In sola lettura.
- **Secondo elemento:** la funzione *setter*. È l'unico modo lecito per cambiare quel valore.
- **Argomento di `useState`:** il valore iniziale, usato **solo al primissimo render**. Nei render successivi viene ignorato, perché React tiene già il valore aggiornato in memoria.

Essendo un array e non un oggetto, i nomi li scegli tu (la posizione conta, non il nome). La convenzione universale è `[qualcosa, setQualcosa]`.

Regola d'oro: non modificare mai lo stato direttamente.

```
titolo = "Nuovo";      // SBAGLIATO: React non se ne accorge  
setTitolo("Nuovo");    // CORRETTO
```

Nota dalla trascrizione: puoi anche scrivere `React.useState(0)` senza importare nulla. Vale per tutti gli hook. Ma la forma con l'import nominale è quella standard.

Regole generali di utilizzo degli Hooks

Definizione. Due regole non negoziabili. Se le violi, React lancia un errore o produce bug silenziosi.

Regola 1 — Gli hook si chiamano solo al livello più alto del componente. Mai dentro `if`, cicli, funzioni annidate o blocchi condizionali.

```
// SBAGLIATO  
  
const Componente = ({ loggato }) => {  
  if (loggato) {  
    const [nome, setNome] = useState(""); // ERRORE  
  }  
};  
  
// SBAGLIATO
```

```

const Componente = () => {
  const gestisci = () => {
    const [x, setX] = useState(0);    // ERRORE
  };
};

// CORRETTO
const Componente = ({ loggato }) => {
  const [nome, setNome] = useState("");
  if (loggato) { /* usa nome qui */ }
};

```

Perché questa regola esiste. React non conosce i nomi delle tue variabili di stato: le identifica **per ordine di chiamata**. Al primo render registra "primo `useState` → 0, secondo `useState` → stringa vuota". Al render successivo si aspetta la stessa sequenza. Se un `if` salta una chiamata, l'ordine slitta e React assegna il valore sbagliato alla variabile sbagliata. Di qui l'errore che vedrai: *"Rendered fewer hooks than expected"*.

Regola 2 — Gli hook si chiamano solo dentro componenti React o dentro altri custom hooks. Non in funzioni JavaScript qualsiasi. È anche il motivo per cui i custom hook devono chiamarsi `useQualcosa`: il nome è il segnale che permette al linter di applicare queste regole.

useState — array come valore di default

Definizione. Lo stato può contenere qualsiasi tipo di dato, array compresi. È il caso tipico delle liste.

```

const persone = [
  { id: 1, nome: "Anna" },
  { id: 2, nome: "Marco" },
  { id: 3, nome: "Giulia" }
];

const ListaPersone = () => {
  const [lista, setLista] = useState(persone);

  const rimuovi = (id) => {

```

```

const nuovaLista = lista.filter((persona) => persona.id !== id);
setLista(nuovaLista);
};

return (
  <>
    {lista.map(({ id, nome }) => (
      <div key={id}>
        <h4>{nome}</h4>
        <button onClick={() => rimuovi(id)}>Rimuovi</button>
      </div>
    ))}
    <button onClick={() => setLista([])}>Svuota</button>
  </>
);
};

```

Il concetto centrale qui è l'immutabilità. Non si modifica l'array esistente: se ne crea uno nuovo.

`lista.push(nuovo);` // **SBAGLIATO: modifica l'array esistente**

`setLista([...lista, nuovo]);` // **CORRETTO: crea un array nuovo**

Perché. React decide se ridisegnare confrontando il *riferimento* del vecchio valore con quello del nuovo (confronto superficiale, `Object.is`). Se modifichi l'array in posto, il riferimento resta identico: React conclude che nulla è cambiato e non ridisegna. Ecco perché si usano `filter`, `map` e lo spread, che ritornano sempre un array nuovo.

Nota anche `onClick={() => rimuovi(id)}`: è il pattern della Lezione 2 per passare un argomento senza eseguire subito la funzione.

useState — oggetto come valore di default

Definizione. Stessa logica, ma con un'insidia in più: aggiornando un oggetto devi ricopiare le proprietà che non cambiano.

```

const Componente = () => {
  const [persona, setPersona] = useState({
    nome: "Dario",
    eta: 30,

```

```

    città: "Barletta"
  });

  const cambiaNome = () => {
    setPersona({ nome: "Marco" }); // SBAGLIATO: età e città spariscono
  };

  const cambiaNomeCorretto = () => {
    setPersona({ ...persona, nome: "Marco" }); // CORRETTO
  };
};

```

Perché. A differenza del vecchio `setState` delle classi, il setter di `useState` **sostituisce** il valore, non lo fonde. Lo spread ricopia tutto l'oggetto e poi la proprietà scritta dopo sovrascrive quella che ti interessa. L'ordine conta: `{ nome: "Marco", ...persona }` non funzionerebbe, perché lo spread riscriverebbe `nome` con il valore vecchio.

Alternativa pratica: usare più `useState` separati invece di un unico oggetto, quando i valori non sono logicamente legati.

```

const [nome, setNome] = useState("Dario");
const [età, setEtà] = useState(30);

```

Realizzare un contatore

Definizione. L'esercizio classico che mette insieme stato, eventi e aggiornamenti multipli.

```

const Contatore = () => {
  const [valore, setValore] = useState(0);

  const reset = () => setValore(0);

  return (
    <>
      <h2>{valore}</h2>
      <button onClick={() => setValore(valore - 1)}>-</button>
      <button onClick={reset}>reset</button>
    </>
  );
};

```

```
<button onClick={() => setValore(valore + 1)}></button>

</>

);

};
```

Funziona, ma ha un difetto nascosto che introduce il punto successivo.

useState — Functional Return (il pattern con la funzione)

Il problema. Prova questo:

```
const aumentaDiTre = () => {
  setValore(valore + 1);
  setValore(valore + 1);
  setValore(valore + 1);
};
```

Aspettativa: +3. Realtà: +1.

Perché. Gli aggiornamenti di stato sono **asincroni e raggruppati** (*batching*). Durante l'esecuzione della funzione, `valore` è una costante congelata al valore del render corrente — diciamo 0. Le tre righe calcolano quindi tutte $0 + 1 = 1$. React applica tre aggiornamenti identici e il risultato è 1.

Definizione della soluzione. Il setter accetta, al posto di un valore, **una funzione che riceve lo stato precedente e ritorna quello nuovo**. React garantisce che ogni chiamata riceva il valore aggiornato dalla precedente.

```
const aumentaDiTre = () => {
  setValore((prev) => prev + 1);
  setValore((prev) => prev + 1);
  setValore((prev) => prev + 1);
};

// Risultato: +3
```

Nella trascrizione questo compare come `set value functional rendering: old value => old value + 1`.

Quando usarlo, come regola pratica: ogni volta che il nuovo stato **dipende dal precedente**. Un contatore, un toggle, l'aggiunta a una lista. Quando invece imposti un valore assoluto (`setNome("Marco")`), la forma semplice va benissimo.

```
// Il toggle corretto, molto comune:
setAperto((prev) => !prev);
```

Questo pattern torna anche in un'insidia pratica: dentro un `setInterval` o un `setTimeout`, la forma semplice legge sempre il valore congelato al momento in cui il timer è stato creato. Solo la forma funzionale funziona.

useEffect — concetti base

Definizione. `useEffect` è l'hook che gestisce i **side effect** (effetti collaterali): tutte le operazioni che avvengono **fuori** dal normale flusso di render del componente.

La trascrizione elenca esattamente i casi: *"cambiare il titolo del documento, settare un event listener, fetchare dati da un'API"*. Aggiungi: timer, sottoscrizioni, scrittura su `localStorage`, log esterni.

La regola che c'è sotto: un componente React deve essere una funzione "pura", cioè deve limitarsi a calcolare cosa mostrare, senza toccare il mondo esterno durante il calcolo. Tutto ciò che tocca il mondo esterno va isolato in un `useEffect`.

```
import { useEffect } from 'react';

const Componente = () => {
  console.log("Componente in rendering");

  useEffect(() => {
    console.log("Eccomi, sono useEffect");
    document.title = "Nuovo titolo";
  });

  return <h2>Ciao</h2>;
};
```

Ordine di esecuzione, che è il punto chiave: in console vedi prima **"Componente in rendering"** e poi **"Eccomi, sono useEffect"**. L'effetto viene eseguito **dopo** che React ha completato il render e aggiornato il DOM. Questo garantisce che l'interfaccia appaia subito, senza essere bloccata da operazioni lente.

Esempio concreto del corso — la notifica stile Facebook:

```
const Componente = () => {
  const [valore, setValore] = useState(0);

  useEffect(() => {
```

```

document.title = `Nuovo messaggio (${valore})`;
});

return (
  <>
    <h2>Valore: {valore}</h2>
    <button onClick={() => setValore((prev) => prev + 1)}>aumenta</button>
  </>
);
};

```

Ogni click aggiorna lo stato → nuovo render → `useEffect` viene rieseguito → il titolo della scheda del browser cambia. È esattamente il meccanismo con cui vedi "(3) Facebook" nella linguetta.

useEffect — utilizzo condizionale

Il problema. Non puoi scrivere questo:

```

if (valore > 1) {
  useEffect(() => { ... }); // ERRORE: viola la Regola 1 degli hook
}

```

Definizione della soluzione. La condizione va messa **dentro** l'effetto, non attorno.

```

useEffect(() => {
  if (valore < 1) {
    document.title = "Nessun messaggio";
  } else {
    document.title = `Nuovi messaggi (${valore})`;
  }
});

```

È un principio generale, valido per tutti gli hook: *l'hook si chiama sempre; è il suo contenuto a essere condizionale.*

useEffect — il secondo parametro (dependency array)

Definizione. `useEffect` accetta un secondo argomento: un **array di dipendenze**. Contiene i valori che, cambiando, devono far rieseguire l'effetto.

Ci sono tre configurazioni, ed è fondamentale distinguerle:

```
// 1. NESSUN array → l'effetto gira dopo OGNI render
useEffect(() => {
  console.log("sempre");
});

// 2. Array VUOTO → l'effetto gira UNA SOLA VOLTA, al montaggio
useEffect(() => {
  console.log("solo la prima volta");
}, []);

// 3. Array CON VALORI → gira al montaggio e ogni volta che uno di essi cambia
useEffect(() => {
  console.log("quando cambia valore");
}, [valore]);
```

Terminologia: *mount* = il componente appare per la prima volta; *update* = viene ridisegnato; *unmount* = viene rimosso dal DOM.

Perché è così importante. Senza array, un `useEffect` che fa una chiamata API e poi salva i dati con `setState` crea un **loop infinito**: l'effetto aggiorna lo stato → lo stato provoca un render → il render riesegue l'effetto → e così via. È l'errore che quasi tutti fanno la prima volta. L'array vuoto lo risolve.

Esempio con dipendenza reale — ricerca:

```
useEffect(() => {
  console.log("Cerco:", termine);
}, [termine]);
```

L'effetto scatta solo quando `termine` cambia, non ad ogni render qualsiasi.

Attenzione alle dipendenze omesse. Se dentro l'effetto usi un valore ma non lo dichiari nell'array, l'effetto continuerà a leggere il valore vecchio (*stale closure*). Il linter di React ti avvisa con `react-hooks/exhaustive-deps`, ed è un avviso da prendere sul serio.

Attenzione anche a oggetti e array nelle dipendenze:

```
useEffect(() => { ... }, [{ id: 1 }]); // gira SEMPRE
```

A ogni render viene creato un oggetto nuovo, con un riferimento diverso. Il confronto è per riferimento, quindi risulta sempre "cambiato". Nelle dipendenze si mettono valori primitivi (`utente.id`, non `utente`), o si memorizza l'oggetto con `useMemo` — argomento della Parte 2.

useEffect — Cleanup Function

Definizione. La *cleanup function* è una funzione che `useEffect` può ritornare. React la esegue **prima di rieseguire l'effetto e quando il componente viene smontato**. Serve a smantellare ciò che l'effetto ha creato.

```
useEffect(() => {  
  // setup  
  
  const timer = setInterval(() => {  
    console.log("tick");  
  }, 1000);  
  
  // cleanup  
  
  return () => {  
    clearInterval(timer);  
  };  
}, []);
```

Perché è indispensabile. Senza `clearInterval`, quando il componente scompare dalla pagina il timer continua a girare a vuoto. Se il componente viene montato e smontato dieci volte, ti ritrovi dieci timer attivi. È il classico **memory leak**.

Il caso degli event listener, che la trascrizione tratta come esempio avanzato:

```
const Componente = () => {  
  const [larghezza, setLarghezza] = useState(window.innerWidth);  
  
  useEffect(() => {  
    const gestisciResize = () => setLarghezza(window.innerWidth);  
    window.addEventListener('resize', gestisciResize);  
  
    return () => {  
      window.removeEventListener('resize', gestisciResize);  
    };  
  });  
};
```

```
}, []);

return <h2>Larghezza finestra: {larghezza}px</h2>;
};
```

Senza la rimozione del listener, React tenterebbe di aggiornare lo stato di un componente che non esiste più, con il relativo warning in console.

Il modo per vedere il cleanup in azione: monta e smonta il componente con un bottone e osserva la console. La sequenza è sempre *cleanup del vecchio effetto* → *nuovo effetto*.

useEffect — Data Fetching

Definizione. Il caso d'uso più frequente in assoluto: recuperare dati da un'API quando il componente compare.

```
const url = "https://api.github.com/users";

const Utenti = () => {
  const [utenti, setUtenti] = useState([]);
  const [caricamento, setCaricamento] = useState(true);
  const [errore, setErrore] = useState(null);

  const getUtenti = async () => {
    try {
      const risposta = await fetch(url);
      if (!risposta.ok) throw new Error("Errore nella richiesta");
      const dati = await risposta.json();
      setUtenti(dati);
    } catch (err) {
      setErrore(err.message);
    } finally {
      setCaricamento(false);
    }
  };
};
```

```

useEffect(() => {
  getUtenti();
}, []);

if (caricamento) return <h2>Caricamento...</h2>;
if (errore) return <h2>Errore: {errore}</h2>;

return (
  <ul>
    {utenti.map(({ id, login, avatar_url }) => (
      <li key={id}>
        <img src={avatar_url} alt={login} width="50" />
        <span>{login}</span>
      </li>
    ))}
  </ul>
);
};

```

I tre stati canonici. Praticamente ogni fetch in React usa questo terzetto: **dati**, **caricamento**, **errore**. Il corso lo ripete in tutti i progetti e nella Parte 2 lo trasformerà in un custom hook riutilizzabile (useFetch).

Due dettagli tecnici importanti:

1. La callback di useEffect non può essere async. Una funzione `async` ritorna sempre una `Promise`, ma `useEffect` si aspetta o `undefined` o la `cleanup function`. Le due soluzioni:

```

// A) funzione esterna, come sopra
useEffect(() => { getUtenti(); }, []);

// B) funzione async dichiarata e invocata dentro
useEffect(() => {
  const carica = async () => { ... };
  carica();
}, []);

```

2. Lo stato iniziale deve avere il tipo giusto. Se iniziizzi `useState(null)` e poi fai `utenti.map(...)` prima che i dati arrivino, ottieni `Cannot read property 'map' of null`. Inizializzare con `[]` evita il crash. È la causa numero uno di schermate bianche.

Return condizionale

Definizione. Un componente può ritornare markup diverso a seconda dello stato. È il modo standard per gestire caricamento, errori e liste vuote.

```
const Componente = () => {  
  const [caricamento, setCaricamento] = useState(true);  
  
  if (caricamento) {  
    return <h1>Caricamento in corso...</h1>;  
  }  
  return <h1>Dati caricati</h1>;  
};
```

L'unico vincolo: il return condizionale deve stare **dopo** tutte le chiamate agli hook, altrimenti in alcuni render alcuni hook non verrebbero eseguiti, violando la Regola 1.

```
// SBAGLIATO  
const Componente = () => {  
  const [a, setA] = useState(0);  
  if (a === 0) return <p>Zero</p>;  
  const [b, setB] = useState(0); // ERRORE: non sempre raggiunto  
};
```

E vale sempre: un componente può ritornare `null` per non mostrare nulla — è lecito e non è un errore.

Short Circuit Evaluation

Definizione. L'uso degli operatori logici `&&` e `||` per decidere cosa renderizzare dentro JSX. Si chiamano "a corto circuito" perché smettono di valutare appena il risultato è determinato.

&& — mostra solo se la condizione è vera:

```
{messaggi.length > 0 && <p>Hai {messaggi.length} messaggi</p>}
```

Se la condizione è falsa, l'espressione vale `false`, e React non renderizza i booleani. Se è vera, vale il secondo operando, cioè il JSX.

|| — valore di ripiego:

```
const [testo, setTesto] = useState("");  
<h2>{testo || "Valore predefinito"}</h2>
```

Se `testo` è una stringa vuota (`falsy`), mostra il secondo valore.

La trappola classica dei numeri — merita attenzione perché è subdola:

```
{lista.length && <Lista />}
```

Se `lista.length` è `0`, l'espressione vale `0` — e React, a differenza di `false`, **renderizza lo zero**. Ti ritrovi uno "0" solitario nella pagina. Soluzione: forzare un booleano.

```
{lista.length > 0 && <Lista />}
```

Ternary Operator

Definizione. L'operatore ternario `condizione ? valoreSeVero : valoreSeFalso`. È l'unico modo per fare un `if/else` dentro JSX, perché — come visto nella Lezione 2 — le graffe accettano solo espressioni.

```
{loggato  
  ? <p>Bentornato, {nome}</p>  
  : <button onClick={login}>Accedi</button>  
}
```

Utile anche per gli attributi:

```
<div className={attivo ? "card attiva" : "card"}>  
<button disabled={caricamento ? true : false}>Invia</button>
```

Ternari annidati: tecnicamente possibili, illeggibili nella pratica. Con più di due casi, estrai la logica in una variabile prima del `return`, o usa un `return` condizionale.

Mostrare e nascondere un componente

Definizione. Il pattern *toggle*, che combina `useState` booleano, evento e short circuit. È la base di modal, sidebar, menu a tendina, accordion.

```
const Componente = () => {  
  const [visibile, setVisibile] = useState(false);  
  
  return (  
    <>
```

```

    <button onClick={() => setVisibile(!visibile)}>
      {visibile ? "Nascondi" : "Mostra"}
    </button>
    {visibile && <Messaggio />}
  </>
);
};

```

Differenza concettuale importante. `{visibile && <Messaggio />}` **rimuove il componente dal DOM**: quando riappare, viene ricreato da zero e il suo stato interno è resettato. L'alternativa CSS lo tiene nel DOM e lo nasconde soltanto:

```

<div className={visibile ? "modal mostra" : "modal"}>

```

La prima è più pulita; la seconda serve quando vuoi conservare lo stato interno o animare l'uscita con una transizione CSS.

Usando la forma funzionale, il toggle si scrive così:

```

setVisibile((prev) => !prev);

```

Form — impostazione generale

Definizione. Un form in React funziona diversamente da un form HTML tradizionale. In HTML puro, l'input conserva il proprio valore e il browser ricarica la pagina al submit. In React entrambe queste cose vanno gestite manualmente.

```

const Form = () => {
  const gestisciSubmit = (e) => {
    e.preventDefault();
    console.log("Inviato senza ricaricare");
  };

  return (
    <form onSubmit={gestisciSubmit}>
      <input type="text" />
      <button type="submit">Invia</button>
    </form>
  );
};

```

`e.preventDefault()` è obbligatorio: senza, il browser ricarica la pagina e tutto lo stato dell'applicazione va perso. In una SPA è devastante.

Nota: mettere l'`onSubmit` sul `<form>` (invece di un `onClick` sul bottone) fa sì che funzioni anche il tasto Invio, che è il comportamento atteso dall'utente.

Input controllato

Definizione. Un *controlled input* è un input il cui valore è determinato dallo stato React, non dal DOM. React diventa la **single source of truth**: l'unica fonte di verità.

```
const Form = () => {  
  const [nome, setNome] = useState("");  
  
  return (  
    <form>  
      <input  
        type="text"  
        value={nome}           // il valore VIENE dallo stato  
        onChange={(e) => setNome(e.target.value)} // ogni tasto AGGIORNA lo stato  
      />  
      <p>Stai scrivendo: {nome}</p>  
    </form>  
  );  
};
```

Il ciclo, che è la cosa da capire davvero:

1. L'utente digita una lettera
2. Scatta `onChange`
3. `setNome` aggiorna lo stato
4. Lo stato provoca un nuovo render
5. L'input riceve il nuovo `value` dallo stato

L'input non "ricorda" nulla da solo: ogni singolo carattere passa attraverso React. Sembra inefficiente, ma è ciò che permette validazione in tempo reale, formattazione automatica, campi dipendenti l'uno dall'altro.

Se metti `value` senza `onChange`, l'input diventa di sola lettura e React ti avvisa in console: il valore è bloccato sullo stato e niente lo può cambiare.

Input controllato vs non controllato. Il non controllato lascia il valore al DOM e lo si legge solo al submit tramite `useRef` (§3.21). React consiglia il controllato come impostazione predefinita.

Esempio di submit di un form

Definizione. Il flusso completo: stato → validazione → azione → reset.

```
const ListaPersone = () => {
  const [nome, setNome] = useState("");
  const [persone, setPersone] = useState([]);

  const gestisciSubmit = (e) => {
    e.preventDefault();
    if (nome) {
      const nuovaPersona = { id: Date.now().toString(), nome };
      setPersone((prev) => [...prev, nuovaPersona]);
      setNome(""); // reset del campo
    } else {
      console.log("Compila il campo");
    }
  };

  return (
    <>
      <form onSubmit={gestisciSubmit}>
        <input type="text" value={nome} onChange={(e) => setNome(e.target.value)} />
        <button type="submit">Aggiungi</button>
      </form>
      {persone.map(({ id, nome }) => <h4 key={id}>{nome}</h4>)}
    </>
  );
};
```

Tre dettagli che il corso sottolinea:

- `setNome( "")` svuota il campo: possibile solo perché l'input è controllato.
 - `[...prev, nuovaPersona]` crea un array nuovo, per l'immutabilità di §3.4.
 - `Date.now()` come id temporaneo: sufficiente in un esercizio, in produzione l'id arriva dal database.
-

Controllare un input con un oggetto

Definizione. Con più campi, dichiarare un `useState` per ciascuno diventa ripetitivo. Si usa un unico oggetto e un unico gestore.

```
const Form = () => {  
  
  const [persona, setPersona] = useState({ nome: "", email: "", eta: "" });  
  
  const gestisciCambio = (e) => {  
    const nome = e.target.name;  
    const valore = e.target.value;  
    setPersona({ ...persona, [nome]: valore });  
  };  
  
  return (  
    <form>  
      <input type="text" name="nome" value={persona.nome} onChange={gestisciCambio} />  
      <input type="email" name="email" value={persona.email} onChange={gestisciCambio} />  
      <input type="number" name="eta" value={persona.eta} onChange={gestisciCambio} />  
    </form>  
  );  
};
```

I due meccanismi da capire:

1. L'attributo `name`. Deve corrispondere esattamente alla chiave nell'oggetto di stato. È ciò che permette a un solo gestore di servire tutti i campi.

2. Le chiavi dinamiche (*computed property names*). La sintassi `[nome]: valore` dentro un oggetto significa "usa il *valore* della variabile `nome` come chiave". Senza le parentesi quadre creeresti letteralmente una chiave chiamata `"nome"`, sempre la stessa.

```
const chiave = "email";  
  
{ [chiave]: "test@test.it" } // → { email: "test@test.it" }
```

```
{ chiave: "test@test.it" } // → { chiave: "test@test.it" } ← sbagliato
```

Versione compatta con destrutturazione e forma funzionale:

```
const gestisciCambio = ({ target: { name, value } }) => {  
  setPersona((prev) => ({ ...prev, [name]: value }));  
};
```

Attenzione alle parentesi tonde attorno alle graffe: senza, JavaScript interpreterebbe { come apertura di blocco invece che come oggetto.